

# ALATI ZA RAZVOJ SOFTVERA

1. nedelja, 18. oktobar 2022.

~ informacije o kursu ~

Ispit (teorija i zadaci) + projekat (izvestaj o primenjenih 10ak alata nad projektom)

~ Uvod u kurs, GCC, BASH, Vim ~

Error ≠ Warnings (greska je nesto sto je standardom definisano kao greska, a sve ostalo je upozorenje, a dobro bi bilo I da su opozorenja zapravo greske)

## Shell

- zsh, bash, python

Osnovne komande:

- mkdir pravi novi direktorijum
- cd premesta nas u zadati direktorijum
- !\$ uzima poslednji argument prethodne linije
- rmdir uklanja zadati direktorijum
- pwd daje informaciju u kom smo direktorijumu trenutno
- pushd pamti direktorijume kroz koje smo dosli
- touch pravi novi fajl
- rm -fr brise sve fajlove iz tog direktorijuma, opcija fr je force recursive, NE !)
- cp -a kopira (klonira) sve, cak i privilegije
- mv pomeranje
- ls -l izlista sve u koloni
- ls -l detaljnije izlistavanje
- ls -lt izlista po redosledu kreiranja (vremenski)
- ll skraceno za ls -l + ls -l
- chmod dodavanje/oduzimanje prava rwxrwxrwx (user, group, others) npr. chmod go-rw
- chown menja vlasnika i grupu za zadati fajl

Osnovne komande za rad sa tekstem:

- echo "Hello, world!" sa navodnicima tekst moze da sadrzi I promenljive
- echo 'Hello, world!' sa apostrofima tekst je samo string
- echo ... > out.txt prebacuje sadrzaj u datoteku out.txt
- ls -lt / | head [-n 5] za pokretanje komande sa desne strane je |, head je za prvih n
- ls -lt / | tail [-n 5] tail uzima sa kraja n

- `tr a-z A-Z` konvertuje sva mala slova u velika
- `tr -c A-Za-z '\n'` menja sve sto nije slovo novim redom (ovako se pravi lista reci)
- **sed** je koristan za lepsi/laksi input:
- `sed 's/ASD/123/'` menja ASD sa 123
- `sed 's/ASD/123/g'` menja **svako** ASD sa 123 (zbog opcije **g**)
- `sed 's/[^ ]* //'` menja bilo sta sto nije razmak praznim stringom
- `wc` prebrojava reci (word count)

- u shell-u je 0 true

- postoji logicko I i ILI ( `&&` i `||` ). Mozemo da ih koristimo npr. ovako:

`touch /root/ime_fajla && vim echo "USPEH"` u znacenju da ako je uspesno otvoren fajl otvori vim i ispise uspeh

`#!/bin/bash` je putanja do interpretera

`if [[ uslov (npr. -e /tmp/... ) ]] ; then` komanda [ ] je komanda test (man test)

....

else

...

fi

- pokretanje shell skripte:

`chmod uog+x script.sh`

`./script.sh`

`for file in /* ; do`

`echo $file` (ovim \$ uzimamo vrednost promenljive)

`done`

`function f() {`

`echo $1 $2` (ovo su argumenti funkcije)

`}`

- komanda `alias` izlistava sve aliase

- aliase mozemo definisati:

`alias ll = 'ls -alh'`

`alias gst = 'git status'`

- komanda `export ime_promenljive = nesto` definise novu promenljivu unutar shell-a

`vim ~/.bashrc` je shell skripta koju mi definisemo

`vim ~/.zshrc`

## 2. nedelja, 25. oktobar 2022.

~ nastavak ~

lokalna promenljiva za jednu komandu se definise:

PAGER=more man test (nema export)

uklanjanje promenljive: unset PAGER

```
for prom in a b c d ; do
```

```
echo $prom
```

```
done
```

 (rezultat je u svakom redu ispisano a b c d)

- kad god pozivamo ili pristupamo svojim promenljivama, koristimo nadovnike

seq 0 10 generise stringove od 0 do 10

```
for i in `seq 0 10` ; do
```

```
echo $i;
```

```
done
```

- **source** (skracenica je . ) ne pokrece novi proces, za razliku od poziva ./script.sh

source script.sh

f(1,2) sada ta funkcija postoji

- izdvajanje stringova iz fajla:

```
sed 's/"[^"]*" /OVDE JE BIO STRING/' script.sh
```

koriscenjem **grep**-a moze lakse:

```
grep '""' script.sh
```

 time dobijemo redove koji sadrze ""

- proverda da li se nalazimo u zeljenom direktorijumu: if [[ "\${PWD}" == "/home/..." ]] ;

- ne postoji fajl .git: ! -e .git

- simbolicki linkovi daju vezu ka necemu: ln -s /...

~ GIT ~

problemi sa rucnim back up-ovanjem svake verzije projekta/koda:

- zauzima se previse memorije

- ne znamo gde je sta izmenjeno, back up-i nisu dovoljno dobro imenovani da reprezentuju sta je u toj verziji uradjeno
- primer naziva back-up-a projekta: bkp20221009-001-implemented-main

problemi sa cuvanjem jednog fajla u kojem se nalaze informacije o izmenama:

- ako slucajno obrisemo taj fajl, gubimo celu istoriju
- ako fajl postane korumpiran, necemo moci da ga otvorimo

problemi pri cuvanju samo izmena izmedju verzija projekta koriscenjem komande **diff**:

- ako izgubimo neki diff izmedju, necemo moci da vratimo sve verzije pre njega
- ako vise ljudi radi na projektu, teska je sinhronizacija izmedju njih i njihovih verzija (programer1 uradi nesto i posalje programeru2 zip-ovan projekat, onda programer2 uradi diff i posalje ime fajla nad kojim je napravio izmene, onda programer1 uradi patch da primeni te izmene na svoju trenutnu verziju projekta, tkd je to naporan posao ukoliko imamo vise programera koji ucestvuju jer onda promene od konkretno nekog programera ostali programeri ne znaju )

~ resenje1: sistem subversion (**SVN**) koristiti centralni server na kojem ce se nalaziti sav izvorni kod, pamte se trenutne verzije projekata i svi diff-ovi na osnovu kojih mozemo da vidimo kako je projekat u nekom trenutku izgledao. Problem je sto konstantna konekcija sa serverom nije moguca zbog interneta tj. ne mozemo nista offline da radimo.

~resenje2: **GIT - sistem za pracenje verzija** treba da bude distribuiran. Svi programeri u lokalu imaju kompletnu istoriju celog projekta. Svaki programer ima svoj racunar na kom ima svoj projekat + ceo repozitorijum gde se nalazi cela istorija tog projekta. Drugi programer isto. Time je omoguceno da svako kod sebe lokalno moze da pravi izmene na projektu, bez konekcije sa serverom. Pored toga, postoji udaljeni server koji je glavni, kojem se sve izmene salju i odakle se sve izmene preuzimaju. Na taj nacin, komunikacija se vrsi samo izmedju servera i konkretnog programera (a ne izmedju programera medjusobno). Takav sistem je GIT.

Do kolizije dolazi kada pokusamo serveru da posaljemo fajl koji je u medjuvremenu (od naseg poslednjeg pull-a ili od kada smo branch-ovali) vec izmenjen od strane nekog drugog programera i push-ovan na server. Kolizije se moraju rucno razresiti.

Git (alat za verzionisanje) ≠ GitHub, GitLab, ... (oni su samo kompanije/projekti/servisi koji nam pružaju usluge GIT-a)

neke komande:

- git init – inicijalizuje git repozitorijum u trenutnom direktorijumu (nije prazan, sadrzi .git sa razlicitim fajlovima (konfiguracionalni fajlovi, baza podataka))

- `git status` – ispisuje koje su izmene napravljene od prethodne verzije (koji fajlovi su menjani, dodati, obrisani, koji nisu dodati u staging area, ako ima commit-ova koliko njih je push-ovano, gde su merge konflikti i slicno)
- `git add filename` – dodavanje izmena u staging area (na taj nacin zakazujemo da ga dodamo u commit) Staging area je deo koji sistem za verzionisanje prati i koji se komituje komandom `commit`, prakticno zakazivanje izmena da budu deo sledeceg komita. Working area su izmene koje nisu premestene u staging area. Bitno: pricamo o izmenama, a ne o fajlovima. Neke izmene nad istim fajlom se mogu dodati u staging area, a druge ostaviti u working area
- `git rm --cached filename` – uklanjanje fajla iz staging area
- `git commit` – ako ne napisemo opis, nece komitovati. Dobra praksa je da se pise naslov, prazan red, poruka o izmenama ukratko, prazan red, detaljniji opis izmena u editoru koji ta komanda otvori.
- `git commit -m "neka poruka"` - pravi novi komit koji ima samo naslov
- `git config --global` – isto sto i menjanje `.gitconfig` fajla (u njemu je moguće definisati aliase. Takodje moze se definisati editor koji se otvara prilikom `git commit` komande: `export EDITOR=ime_editora`) u home direktorijumu repozitorijuma

Fajl `.gitignore` služi da u njemu definisemo one fajlove koje sigurno ne zelimo da commit-ujemo.

- `git log` – cela istorija commit-ova, svaki commit u sledecem obliku: commit id- "kobasica" heksadekadnih cifara za identifikaciju commitova. Koriste se hash sume u koje se ukljucuje korisnik koji je kreirao commit, prethodni komit, datum kreiranja, sadrzaj fajla itd. da se generise jedinstveni commit id.
- `git diff` – izmene od poslednjeg commita. Napomena: ovo se odnosi na izmene koje su u working area, kada se jednom doda fajl u staging area komandom `git add` ne prikazuje se vise uopste preko diff
- `git diff id_commita` – prikazuje razlike od zadatog komita do trenutne verzije (SVE izmene, bez obzira da li su u working ili staging area). Ne mora ceo id\_commita da se pise, dovoljno je prvih nekoliko (6) cifara, ostale ce git dopuniti sam. U slucaju da ima vise komitova cije su to pocetne cifre, bira se skoriji
- `git diff HEAD` – diff od poslednjeg commita. Moze i `git diff HEAD~1`, `git diff HEAD~2`

Glavna grana (master) je glavna razvojna verzija naseg programa. Grane sluze kada zelimo da pravimo vise razlicitih stvari, da koristimo po 1 granu za svaku stvar. Dobra praksa je da za svaku novu funkcionalnost, napravimo novu granu.

Grane su pokazivaci, neophodni da bi master grana uvek bila funkcionalna.

- `git branch ime_grane` – pravi novu granu. Dobra praksa je da ime grane sadrži naziv kreatora, npr. `ivan/bug123-output-doesn't-work`. Pravljenjem nove grane, kao da smo trenutno stanje programa smestili u tu granu. Ukoliko napravimo neke izmene na master grani, one neće biti vidljive i sa novokreirane grane. Takođe, izmene koje mi pravimo na našoj grani, neće biti vidljive na glavnoj.
- `git branch -d` - briše granu koja je spojena sa merge
- `git branch -D` - briše granu koja nije spojena
- `git checkout ime_grane` – prelazimo sa trenutne grane na granu `ime_grane` (u modernom gitu koristi se: `git switch`)
- `git merge ime_grane_koji_zelimo_da_ubacimo_u_master` – spaja master granu na kojoj smo trenutno sa granom koju smo naveli

### 3. nedelja, 1. novembar 2022.

~ nastavak GIT-a ~

- `git clone url` – klonira git repozitorijum sa servera u naš izabrani direktorijum, tj. pravi lokalni repozitorijum kod nas na računaru. Veza između ta 2 (glavni repozitorijum na serveru i naš lokalni) je ostvarena tako što naš repo ima pokazivac (koji se zove `origin`) na glavni repo.
- `git remote add personal url` – dodaje pokazivac (koji se zove `personal`) sa našeg repo na taj koji smo prosledili kroz `url`
- `git remote -v` – izlistavaju se svi remote pokazivaci koje imamo. (u našem slučaju, 2 remote: `origin` i `personal` od kojih svaki ima 2 instance, 1 za `fetch`, drugu za `push`) Ovo služi da ako npr. imamo projekat koji se radi širom sveta, a mi želimo da dovlacimo promene samo koje se prave u Beogradu, onda nam ta 1 instanca za `fetch` služi baš za to, a `push` ćemo raditi na glavni zajednički server.
- `git log --oneline` - izlistavaju se komitovi, svaki u zasebnoj liniji (`id`, zaglavlje)
- `git log --oneline --all --decorate --graph` – za lepši prikaz istorije logova + stablo grana (posto je komanda velika, često se za nju definiše globalni shell alias: `alias glol='git log --oneline --all --decorate --graph'` ili u `.gitconfig` fajlu u sekciji za aliase napisemo: `lol = log --oneline --all --decorate --graph` i onda pozivamo u terminalu sa `git lol`)
- `git whatchanged` – prikazuje šta je izmenjeno i koji fajlovi su izmenjeni u komitovima
- `git log --putanja_do_fajla_koji_nas_zanima` – izlistava samo commitove koji su vezani za taj fajl
- `git branch --all` - izlistava sve postojeće grane

Tagovi (tags) su read-only pokazivaci na konkretnu verziju u istoriji naseg projekta, koji se nikad ne menjaju (nikad necemo popraviti bag u tom tagu) To je kljucna razlika izmedju tagova i grana, grana služi da nastavimo sa razvojem odvojeno od glavne verzije programa, a tagovi samo da damo ime konkretnom komitu i da taj tag uvek pokazuje na taj konkretan komit.

- `git tag` – izlistava sve postojece tagove

- `git checkout --track -b ime_nase_grane ime_udaljene_grane` – nasa lokalna grana koju cemo napraviti treba da prati tu neku udaljenu granu, znaci nasa grana ce se sinhronizovati sa tom nekom udaljenom granom (kad budemo radili push/pull hocemo da push-ujemo i pull-ujemo na tu udaljenu granu)

- `git push personal`

`ime_nase_grane` : `ime_kako_zelimo_nasa_grana_da_se_zove_na_udaljenom_serveru`  
(personal je `ime_servera_na_koji_saljemo` )

Za istu svrhu mozemo da koristimo: `git push origin ime_na_udaljenom` ako smo pre toga uradili `git checkout --track -b`

- `git push origin ime_nase_grane` – na taj nacin nasu granu saljemo na server origin (u terminalu dobijamo poruku da ako zelimo od te grane da napravimo merge request idemo na taj url, kada kliknemo otvara nam se New merge request na GitLab-u)

- `git fetch` – povlaci sve izmene ali ne primenjuje

- `git pull` = `git fetch` + `git merge`

- `git merge ime_grane_koju_ubacujemo_u_nasu_trenutnu` – sve komitove iz grane koju navodimo prenosi u novonastalu granu koju spaja sa onom u kojoj smo trenutnu. Tada moze da dodje do merge conflicts ukoliko postoje fajlovi koji su izmenjeni u obe grane, pa se to rucno mora razresiti i komitovati.

- `git rebase nova_grana` - sve komitove koji su se desili na master grani posle pravljenja te grane hocu da prebaci na vrh te nove grane (prakticno pravi nove komitove na vrh te neke nove grane). (rebase treba izbegavati ali moze da se koristi `git rebase master`)

razlika izmedju merge i rebase:

merge pravi 1 commit (od 2 koja spaja) i istorija osta ista nadalje. Graf je haotican.

rebase sve izmene se primenjuju na master (aktuelnu verziju). Dodajemo nove komitove i novu istoriju. Graf je linearan.

- kretanje po istoriji:

- `git checkout` (to koristimo ako zelimo samo da pogledamo tu verziju, a ne da je razvijamo)

- git branch (ako zelimo da nastavimo ravijanje)
- git reset HEAD~1 time resetujemo poslednji commit I vracamo se na 1 unazad (brise se samo informacija o commitu, sadrzaj ostaje)
- git reset --hard HEAD~1 brise I sadrzaj prethodnog commita
- git revert – ako zelimo da ponistimo neke izmene tj. da neki komit napravimo inverznim
- git stash – ponistava izmene ali ostavlja ih za slucaj da nam mozda kasnije zatrebaju (poslednja se vracia sa git stash pop, a moze I: git stash pop stash@{1\}, a izlistavamo sa git stash list)

• git cherry-pick – kada zelimo samo 1 commit da prebacimo I na druge grane (korisno kada npr. imamo isti bag na vise grana, a popravimo ga na jednoj, hocemo da bude popravljen na svakoj) U toj kopiramo sha kod komita u kom je popravljen bag, prebacimo se na tu drugu granu na kojoj je bag popravljen I onda uradimo: git cherry-pick sha\_kod. To ce da prenese na novu granu isti taj komit (isti sadrzaj tog komita, ali novi jedinstveni id)

• git commit –amend – modifikovanje poslednjeg komita (zapravo pravi novi komit, a uklanja stari, taj stari ne pripada vise nijednog grani)

• git show commit\_id – prikazuje sadrzaj datog komita

• git cat –file –t –s – izvlacimo sadrzaj fajla, -t je tip objekta, -s je velicina fajla

autor (osoba koja je stvarno napravila komit) ≠ comiter (osoba koja ima privilegiju da postavi nas komit)

- sve sto ikada uradimo/dodamo u repozitorijum, bice u objects direktorijumu

- Kada radimo git pull moze doci do merge conflicts problema. Git push radimo pod uslovom da smo uradili git pull I razresili sve kolizije.

## 4. nedelja, 8. novembar 2022.

~ nastavak GIT-a ~

• git bisect start – zapocinje algoritam binarne pretrage (npr. desi se bag u nekom komitu ranije I zelimo da otkrijemo u kom tacno. Mogli bismo linearno da trazimo redom od poslednjeg komita koji nema bag, ali efikasnije je da koristimo binarnu pretragu, tj. polovimo intervale)

• git bisect good id\_commita – time kazemo da bag nije postojao u tom komitu

• git bisect bad master – u ovom komitu (master) postoji bag

- git bisect reset – izlazimo iz algoritma bisekcije (nasli smo komit u kojem je nastao bag)
- git worktree – kao da pravi ceo klon repozitorijuma na nekoj drugoj lokaciji (npr. git worktree add ime\_direktorijuma ime\_nove\_grane – time smo se pozicionirali u dati direktorijum na bas tu datu granu). Razlika izmedju ovoga i git clone je sto git worktree ne zauzima duplo mesta (deli se .git direktorijum)
- git worktree remove putanja\_do\_grane – zavrшили smo rad na toj grani
- git worktree list – izlistavaju se sve worktree grane
- git submodule add url\_do\_repozitorijuma – kada u nase projekte zelimo da ukljucimo i neke druge projekte (da generisemo veze ka njima, te veze se zovu submodule tj. podmoduli (npr. biblioteke))
- git submodule update --init – kada kloniramo neke repozitorijume koji imaju submodule, moramo rucno da kloniramo i njih tom komandom (a ako ti submodule imaju svoje submodule, onda dodajemo opciju --recursive)
- git grep abc – primenjuje grep na samo one fajlove koji su poznati git-u tj. pretrazuje samo izvorni kod naseg projekta (ne pretrazuje .git ili build fajlove)

## 5. nedelja, 15. novembar 2022.

### ~ GitLab ~

- git push origin ime\_nase\_grane – na taj nacin nasu granu saljemo na server origin (u terminalu dobijamo poruku da ako zelimo od te grane da napravimo merge request idemo na taj url, kada kliknemo otvara nam se New merge request na GitLab-u)
  - Description treba da sadrzi informacije o tome:
  - sta je promenjeno
  - razlog zasto je promenjeno
  - nacin na koji je nesto uradjeno
  - objasniti alternativne pristupe i zasto je nas odabrani pristup bolji od njih
  - ukoliko ta nasa izmena popravlja neki bag, obavezno staviti link do njega ( npr. Fixes #123 a npr. Fixes: #1 daje referencu ka zadatku pod rednim brojem 1)
    - Assignee – ko treba da uradi taj zadatak
    - Reviewer – jedna ili vise ljudi od kojih trazimo da pogleda nas kod (oni imaju opciju Approve da odobre nas kod ukoliko je sve dobro)
    - Milestone – do kada ovaj zadatak treba da bude uradjeno
    - Labels – kao tag da nas lakse odvede do nekog manjeg dela
    - Merge options:

- Delete source branch when merge request is accepted. (automatski brise granu koju smo napravili zbog tog merge request-a)
- Squash commits when merge request is accepted. (sve komitove koje smo napravili ce pretvoriti u 1 I to ce se na master grani prikazati kao 1 komit umesto cela istorija komitova, preporuka: ne )

Pored Overview, postoje I kartice: Commits (istorija komitova), Pipelines (koji testovi su prosli, a koji ne) I Changes (najbitnije, spisak izmena u samom kodu). Ako neke izmene koje se tu nalaze u kartici Changes, smatramo da nisu deo tog merge requesta, imamo opciju da dodamo komentar (npr. This doesn't belong to this MR). Tada nam se otvaraju 2 opcije: Start a review I Add comment. Razlika izmedju njih je sto ako znamo da cemo imati vise komentara koristimo start a review, a ako smo sig da ce to biti samo taj jedan komentar I tu zatvaramo analizu, onda koristimo add comment. (preporuka: start a review). Kada tek dodamo te komentare pise "Pending" a to znaci da nismo jos zvanicno poslali autoru. Da bismo poslali, kliknemo "submit review". Autor sada moze da razresi problem I klikne "Resolve thread" ili da odgovori tekstom.

Tek nakon sto nama odredjeni klikne "Approve", mi mozemo da kliknemo "Merge when pipeline succeeds".

U nekim drugim sistemima, ne postoji dugme "Approve" vec score (-2, -1, 0, +1, +2). Tada ako nam daju +2 poena, znaci da mozemo da merge-ujemo. A npr. -1 znaci potrebne su neke izmene pre nego sto predje u +2, a npr. -2 znaci da to nikada nece biti spojeno sa master granom, totalno pogresno.

Sta sve treba da se proverava u **code review**-u ( u slucaju da smo mi reviewer-i)?

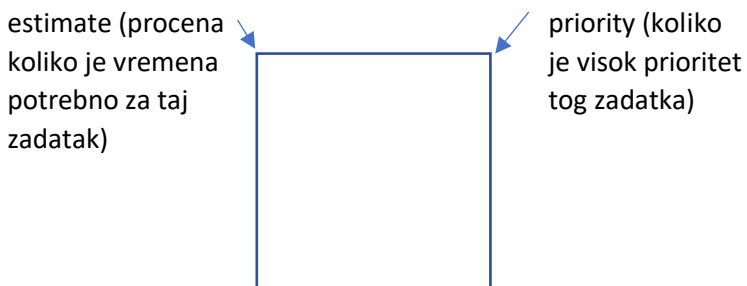
- dizajn softvera
- imenovanje funkcija I promenljivih
- formatiranje (kroz ceo projekat treba da bude 1 format, npr. snake\_case)
- slozenost (vremenska I memorijska)
- korektnost konkretnog programskog jezika (npr. da li na ispravan nacin koristimo C++)
- pokrivenost testovima
- azuriranje dokumentacije
- still pisanja (npr. gde se stavljaju { } )
- mrtav I zakomentaran kod
- primenljivost na ranije verzije
- citljivost
- svrha koda – da li nam to uopste treba

Issues: List, Board, Milestones (zadaci koje treba uraditi I njihova organizacija)

Proces zadataka prolazi kroz 6. razlicitih faza:

1. Backlog (tu se nalaze svi zadaci koji su planirani da se rade u buducnosti)
2. Selected (koji zadaci iz 1. ce se u sledecem periodu raditi)
3. In Progress (ovde su zadaci na kojima se trenutno radi I tu svaki zadatak mora da ima assignee polje odnosno ko je zaduzen za rad na tom zadatku)
4. In Review (zadaci koji su u procesu "pregledanja" tj. analize)
5. Done (zadaci koji su završeni, ovde je posao developer-a gotov)
6. Merged (zadaci koji su uspesno prosli code review I merge-ovani na master granu)

Zadaci se pojedinačno mogu ocenjivati I na sledeci nacin preko kvadrata:



- u praksi se cesto za poene koristi Fibonacijev niz. Svaki zadatak ima neki broj Fibonacijevog niza 1 2 3 5 8 13. Najlakši zadatak ima tezinu 1, a najteži 13.

6. nedelja, 29. novembar 2022.

~ GDB ~

Najveća mana koriscenja printf komande za otkrivanje gresaka u programu je neoptimalnost (linearna pretraga). Kada trazimo bagove, mi ustvari trazimo razlike izmedju naseg razmisljanja (modela koji smo stvorili za taj program) I stvarnog ponasanja tog programa. Ako smo napravili greske u modelu programa (neku logiku lose implementirali) takve greske debager ne moze da otkrije. Jedine greske koje se mogu otkriti debagerom su one gde se implementacija razlikuje od modela koji smo napravili. U slucaju velikih funkcija (mnogo linija koda) bolje od linearne pretrage je naravno binarna pretraga. Podelimo funkciju na 2 dela I gledamo u kojoj polovini se desila greska I to ponavljamo dok ne dodjemo do jako malog dela koda koji mozemo da pretrazimo linearno. Medjutim problemi sa tim su naporno rucno pisanje printf-ova I jos veci problem, prilikom podele na vise delova, za svaki deo moramo da rekompiliramo ceo program

(može da bude jako sporo). Prednost ispisivanja printf-ova je ta ako imamo višenitni sistem, možemo na lep način da vidimo u kojoj se niti trenutno nalazimo. Za printf-ovanje nije potrebno nešto više znanja od običnog programiranja, ali nije na svakom uređaju moguće izvršiti printf-ovanje (npr. ne postoji ekran na kom bi se ispisala greska, u slučaju frizidera, mašina za sudove itd).

- Situacije u kojima programeri najviše koriste GDB je kada dodje do curenja memorije (segmentation fault). Postoje 2 načina za debugovanje: 1) iz gdb-a pokrenuti našu aplikaciju 2) za već pokrenutu aplikaciju zakacimo GDB (ovo se danas skoro uopšte ne koristi ali da bi se to omogućilo potrebno je poslati 0 u fajl ptrace\_scope odnosno izvršiti komandu `echo 0 > /proc/sys/kernel/yama/ptrace_scope` u terminalu, zatim je potrebno proveriti koji je PID (process ID) aplikacije koju želimo da debugujemo, komandom: `ps aux | grep ime_fajla`, zatim pokrenuti: `gdb ./src/ime_fajla`, zatim `attach PID`. Nakon toga, program se zamrznuo odnosno omogućeno je definisanje break point-a, naredbom `cont` možemo nastaviti izvršavanje. Naredba `bt` (back trace) izlistava stack trace svih poziva funkcije, a možemo i samo prvih 10 sa `bt 10`. Sa naredbom `frame broj_frejma` možemo da definisemo koji nas frejm interesuje )

- Definisanje break point-a je naredbom: `break ime_klase::ime_funkcije`. Ako želimo da se napravi break point samo ako je neka naša promenljiva uzela konkretnu vrednost onda može: `break ... if prom = vrednost`.

- Ako želimo da napravimo break point na svakoj funkciji koja zadovoljava neki regularni izraz: `rbreak ime_klase::.*` (uzima sve članske funkcije date klase)

- Naredbom `info breakpoints` možemo da vidimo (izlistamo) sve break point- e koje imamo

- Ako želimo da nam break point ne zaustavlja program već samo radi kao printf: `set logging redirect on` i onda `set logging on`. Tada definisemo komande koje želimo da se izvrše za svaki break point:

commands

> bt 3

> print this

> cont

> end

I pokrenemo program sa `run`.

- Postoji opcija i da debugiramo tako što se možemo vratiti i unazad (a ne samo ici napred sa next). Potrebno je “uključiti snimanje” komandom: `target record-full`, zatim definisemo break point kada ćemo da završimo sa snimanjem: `break ime_klase::ime_fje` i onda `cont`.

- **Core file** je fajl koji sadrži trenutno stanje programa kada je program pukao.

- Za analizu sadržaja procesorskih registara koristimo: `tui` (text-based user interface) `reg general` (prikaz registara opšte namene)

- U situaciji kada mi očekujemo da x ima vrednost 42, a desi da to nije slučaj, možemo da ispravimo program i rekompajliramo ispočetka, ali umesto toga možemo da kažemo gdb-u da

promeni vrednost sa: set x = 42 (ovo se radi dinamicki, zaustavi se program, promeni vrednost i nastavi izvršavanje sa novom vrednoscu). Takodje, ako u nekom trenutku zelimo da pozovemo neku funkciju koju nas program inace ne poziva, mozemo: call this->nekaFunkcija(42).

## 7. nedelja, 6. decembar 2022.

### ~ Paketi ~

Menadzeri paketa (package managers) sluze kada nam osnovne biblioteke nisu dovoljne (npr. hocemo da se iscrta kocka koja se okreće u prostoru). Tada cemo koristiti npr. OpenGL biblioteku. Postoji pkg-config, na putanji usr/.../fajl.pc se nalaze fajlovi sa ekstenzijom pc (package config) u kojima se nalaze flag-ovi koje kompajler treba da ukljuci da bi koristio izabranu biblioteku. (-I govori gde se nalaze flegovi, -l koje biblioteke linkujemo i -L gde se nalaze te biblioteke). U praksi se obicno pravi novi direktorijum 3rdparty i onda u njega kopiramo celu biblioteku i tako za svaku biblioteku koja nam zatreba. Takodje, treba onda dodati u CMake da se ukljuce i te nove biblioteke kao deo naseg projekta, to je problem jer onda moramo svaki put da kompajliramo sve i svaki clan tima mora da kopira cele biblioteke, sto je dosta neefikasno.

- Vecina programskih jezika dolazi sa svojim menadzerima paketa, npr. pip za python, stack za haskell, ali za C++ ne postoji jedan standardni menadzer paketa.

- Postojanje i koriscenje biblioteka nam omogucava ponovno koriscenje koda (code reuse)

- C++ nema jednu kompaniju iza sebe, postoje razlicite implementacije C++-a => postoje razliciti kompajleri: gcc, clang, msvc, intel, ... Zato je i tesko napraviti jedan package manager koji ce raditi za sve te verzije.

Samim tim i biblioteke mogu da budu kompajlirane razlicitim build sistemima (CMake ima 40% trzista, GNU make 30%, qmake 7%)

=> package manager koji bi trebao da radi sa C++-om bi trebao da brine o: razlicitim kompajlerima, razlicitim platformama i razlicitim build sistemima.

Bilo bi lepo da postoji **reproducible builds**. Ideja je ako imamo podesen build sistem, package manager i sve ostalo, da imamo jednu jedinu komandu koja ce garantovati da na kom god racunaru da smo, pod kojim god OS, da ce uvek da kompajlira nas kod korektno i da ce na svim tim sistemima da ga kompajlira isto (da imamo iste mogucnosti, da su sve zavisnosti povucene kako treba bez obzira na kojoj se platformi nalazimo).

Kod package manager-a (ali i kod mnogih drugih koncepata) moze doci do problema dijamanta (**diamond problem**): Imamo program A, koji koristi biblioteke B i C, koje obe zavise od biblioteke D. Ako biblioteka B zahteva odredjenu verziju biblioteke D, a biblioteka C zahteva neku jos noviju verziju biblioteke D. Ukoliko su te 2 verzije kompatibilne nece biti problema jer ce se uzeti novija,

ali ako nisu kompatibilni, između B i D će doći do kolizije. To je nešto što package manager mora da razredi (a većina bi samo javila da ne može to da razredi i da mi smislimo šta ćemo da radimo).

Sistemski package manager-i: apt, jam (JavaScript package manager) i mnogi slični razrešavaju prethodno opisane probleme veoma lepo (npr. problem dijamanta ne može ni da se desi jer se uglavnom definišu samo jedne verzije određene biblioteke). Takođe, što se tiče problema različitih kompajlera obično je većina kompajlirana gcc-om ili clang-om, a build sistem se koristi sam apt. Što se tiče reproducible builds uglavnom postoji.

Trenutno najpopularniji package menadžeri (za instalaciju potrebnih biblioteka) za C++ su: **Conan** i **vcpkg**. Postoji i **nix** (ali nije vezan samo za C++, više za funkcionalne jezike, kasnije o njemu).

Conan implementiran u python-u, instalira se sa: `pip3 install --user conan`

Conan omogućava da se koriste različiti build sistemi (ne forsira samo CMake).

Sve vezano za Conan stavljamo u `conanfile.txt` (to nije običan txt fajl, ima svoju posebnu sintaksu). Unutar `conanfile.txt` definišemo 2 sekcije: `[requires]` su biblioteke koje zahtevamo da koristimo u projektu i `[generators]` koji se odnosi na to koji build sistem želimo da koristimo (CMake). Da bismo videli koje sve verzije biblioteka postoje: `conan search --r all poco` (opcija `--r all` znači da se pretražuju svi repozitorijumi). Da bismo lokalno instalirali sve zavisnosti iz tog fajla: `conan install ..` (pored te same biblioteke koju smo naveli, instaliraju se i sve njene zavisnosti, zatim zavisnosti od tih zavisnosti tj. celo drvo zavisnosti)

Komanda `conan info ..` nam daje sve zavisnosti

Moramo na neki način CMake-u da kažemo da Conan postoji, zato u `CMakeLists.txt` pišemo: `include(${CMAKE_BINARY_DIR}/conanbuildinfo.cmake)`

`conan_basic_setup()` // nakon ovoga možemo da koristimo sve te nove biblioteke

`add_executable(prog main.cpp)`

`target_link_libraries(prog ${CONAN_LIBS})`

- Karakteristike Conana:

- implementiran u python-u (lakši za skriptovanje od `vcpkg-a`)
- decentralizovan (ukoliko nam nisu dovoljni glavni repozitorijumi, možemo da napravimo svoje i odlučujemo koje servere hoćemo da koristimo a koje ne)
- postojanje glavnog repozitorijuma (apt). To znači da postoji taj 1 glavni "sigurni" repo koji garantuje da sve biblioteke koje se nalaze na njemu će međusobno biti kompatibilne (njihove verzije). Tako da ako koristimo glavni repo, ne možemo da dodjemo do problema dijamanta.

- Mane Conana:

- izmena `CMakeLists.txt` (moramo da obavestimo build sistem (CMake) da Conan postoji)

**VCPKG** (Microsoft-ov package manager)

Primarna platforma su Windows-i (logično)

Implementiran je u C++-u, uz CMake fajlove

Primarno je napravljen da radi sa CMake-om i ni sa čim drugim.

Ne postoji fajl u kojem definisemo zavisnosti, vec potrebne biblioteke instaliramo sa npr. vcpkg install poco

Takodje ovde ne definisemo verzije. Ideja je da u celom repozitorijumu postoji jedna jedina verzija i da ona radi dobro sa svim drugim. (centralizovan je, to je losije nego Conan)

Prednost je sto ne moramo da menjamo CMakeLists.txt fajl ali zato prilikom pokretanja CMake-a moramo da dodamo fleg: cmake .. / -DCMAKE\_TOOLCHAIN\_FILE= /putanja do fajla vcpkg.cmake  
Prednost toga je sto ne moraju svi clanovi tima da koriste isti package manager (za razliku od Conana gde smo u CMakeLists.txt morali da kazemo koji package manager koristimo i onda svaki clan tima prilikom kloniranja ima taj isti CMakeLists.txt fajl)

## NIX

Ideja je da ima dobre strane oba (i Conan-a i vcpkg-a) tj. da bude package manager koji moze da bude sistemski package manager (kao sto se vcpkg ponasa, kao sto je apt) ali i da dozvoljava instalaciju razlicitih verzija biblioteka u zavisnosti od potrebe programa.

Cilj je da bude:

- **cist** (pure) u smislu da instalacija necega utice na promenu ponasanja nekog drugog paketa. (npr. imamo program koji zavisi od konkretnog binarnog fajla neke verzije, azuriranje sistema ne treba da utice na ponasanje tog programa) Zato cemo imati direktorijum nix/store u kojem ce se nalaziti svi paketi (sve verzije) koje su ikada instalirane i onda ce za konkretan fajl uvek moci da procita tacnu verziju koja treba). Svaki paket ima svoju hes sumu (ime direktorijuma)
- **deterministicki sistem** (jednom napravljen paket, vecno postoji)
- **composable** (da ga je lako komponovati)

Prednost:

- **reproducible builds** (NIX garantuje, a Conan i vcpkg ne u potpunosti)

Postoji i operativni sistem nixOS, baziran na Linux-u u kome je ideja da jedini package manager bude nix. Lako se instalira: nix install i definisu se paketi tj. zavisnosti koje zelimo. To je dobro u praksi ako zelimo da svaki clan ima identicne verzije paketa, potrebno je samo da svaki clan instalira kod sebe. Takodje, ovo moze da bude korisno kada se prave Build farme.

Informativno, istraziti: appimage, flatpack, snap u kojima su zavisnosti ukljucene u sam izvrsni fajl.

Poredjenje: sistem **apt** (dovlaci potrebne biblioteke sa nekog centralnog repozitorijuma) i sistemi **nix**, **conan** (veziju izvrsni fajl sa konkretnom verzijom biblioteke): Ako biblioteka sadrzi neki bag, time sto apdejtujemo sistem, apdejtovali smo i biblioteku (mana kod sistemskih npr. apt jer onda neki program samo moze da prestane sa radom)

A kod nix, conan-a ko se apdejtovala biblioteka, nasa verzija koda koristi staru, pa nece prestati sa radom. Ogromna prednost sistemskih (apt) je sto se non stop instaliraju poslednje verzije bitnih glavnih biblioteka (npr. OpenSSL), ali opet u slucaju velikih bagova to je njihova mana.

=> Package manager-i su veoma lepa stvar da olaksaju programerima posao, ali imaju dosta ozbiljne propuste: sigurnost i azuriranje samih zavisnosti.

## 8. nedelja, 13. decembar 2022.

### ~ Sanitajzeri ~

Stek okvir: lokalne promenljive, argumenti funkcija, privremeni rezultati, pokazivaci za return  
Stack smashing je pojava da mozemo da predefinisemo izlazni pokazivac iz stek okvira I da skocimo na bilo sta I onda se nece nastaviti izrsavanje funkcija koje se nalaze ispod, tj. nece se vratiti na njih.

Stek kanarinac (stack canaries) - pored pokazivaca gde treba da se vratimo se nalazi neka vrednost (na pocetku je to neka nasumicna vrednost) I prilikom izlaska iz funkcije ce se proveravati da li se ta neka nasumicna vrednost promenila ili nije. Ako je vrednost promenila, OS "ubija" nas program, ako se vrednost nije promenila sve ok.

**Shadow stack** – ili drzimo sve bitne informacije kopirane na jos jednom mestu (iz steka) ili samo neke glavne, najbitnije informacije kloniramo na pomocni stek. Prednost je sto posle nekih funkcija mozemo da proverimo da li se stvari razlikuju na glavnom steku I shadow steku. Ako se razlikuju, "ubijamo" program, a ako se bitne stvari ne razlikuju program nastavlja sa izrsavanjem. (sve sto cuvamo na tom shadow steku nije kopirano (da ne bi dodatno zauzimalo memoriju) vec se cuva u vidu pokazivaca). Znaci, pored glavne memorije imamo shadow memory kao nekakav klon glavne, koji cuva samo bitne infomacije koje posle mozemo da koristimo.

=> sve su to stvari koje mozemo da ukljucimo prilikom kompilacije naseg programa (kanarinac je podrazumevano ukljucen)

Program koji se prevodi iako ima bag:

```
int xs[100] = {-1}      //ovde je nulti element -1, a ostali elementi su nule
int main(){
return xs[110];
}
```

U takvim slucajevima, mozemo da koristimo alat Valgrind ili aktiviramo **Address Sanitizer (ASAN)**, tako sto u Makefile-u ukljucimo fleg: -fsanitize=address I tada vec dobijamo upozorenje tj. poruku da pristupamo elementu koji ne postoji. Takodje taj fleg dodajemo I u procesu linkovanja tj. pravljenja objektnog fajla u Makefile-u. Bude ispisan detaljan prikaz delova memorije (00 znaci memorija koja stvarno jeste zauzeta nasim nizom, f9 je global redzone (memorija koja ne pripada nikome), f3 je stack right redzone (isti kod samo smo deklaraciju xs prebacili u main funkciju)).

Sanitajzeri se ne ukljucuju u projekat (u zavrshu verziju za klijenta) vec ih koristimo jedino kada mi za sebe testiramo. Jedino kada je korisno klijentu poslati I sanitajzer u projektu je ako sumnjamo da postoje neki problemi ili ako su performanse softvera mnogo manje bitne od bezbednosti.

sledeci kod daje fd a to je kada pristupamo necemu sto je bilo alocirano na hipu a u medjuvremenu oslobodjeno, taj bag se zove **use after free**

```
int main(){
```

```

int *xs = new int[100];
delete [] xs;
return xs[110];
}

```

U sledecemo kodu kompajler ne vidi problem, pristupa samo alociranoj memoriji bez elemenata (standardna biblioteka stdlib moze da vidi problem, zato stavimo fleg `-stdlib=libc++` u Makefile, takodje I linkeru)

```

#include <vector>
int main(){
std::vector<int> xs{1,2,3};
xs.reserve(10);
return xs[3];
}

```

- Kako ove stvari uopste rade?

Ideja je da shadow memorija mapira originalnu memoriju samo umesto stvarnog sadrzaja, shadow memorija oznacava da li je celija validna ili nevalidna. Crvenom ce da oznacava odredjeni broj celija post-alocirane memorije I pre-alocirane memorije. I onda kada neko za neki blok memorije zatrazi prostor, nece uzimati zeleni deo naravno, ali ni crveni. Ti crveni delovi oko zelenog dela nazivaju se **poison** memory. Kada se pozove free, oslobadja se originalna memorija, a u shadow memoriji se taj deo oznacava nekom novom bojom, ali svakako ostaje poison. Malloc mora da izbegava blokove memorije koji su skoro bili oslobodjeni, tj. da se trudi da njih sto kasnije iskoristi jer ako dodje do бага use after free, ne bi se taj bag detektovao jer bi na toj lokaciji bilo nesto drugo.



to je heap alokacija

Ako imamo stack alokaciju, iako stek ne koristi malloc, na slican nacin se blokovi memorije oznacavaju I pakuju.

Razlog zasto se sanitajzeri ne ukljucuju u zavrshi projekat je taj sto kompajler usporava izvorsavanje programa time sto svaki put radi proveru da li je memorija kojoj zelimo da pristupimo poison ili ne.

Nepotrebno je da za svaki bajt originalne memorije, mi zaista obelezavamo 1 bajt shadow memorije. U praksi se za svaki red od 8 bajtova iz originalne memorije, u shadow memoriji pamti samo 1 bajt u koji se smesta vrednost promenljive i (i-ti red originalne memorije). Znaci, shadow memorija ce biti 8 puta manja od originalne memorije.

Najbitnije znati: kompajler generise neke dodatne instrukcije koje proveravaju razne stvari, zameni normalne operacije malloc I free nekim drugacijim I dobijemo dosta dobre provere za vecinu problematicnih situacija koje mozemo da imamo sa dinamicnom ili statickom memorijom. Koncept shadow memorije je mapiranje originalne memorije uz dodavanje dodatnih informacija. Mali problem sanitajzera je to sto nas program onda nije samostalan vec mora da koristi neke druge biblioteke (a ako one nisu kompajlirane sa sanitajzerom) moze doci do problema da se bagovi ne detektuju.

**Thread sanitizer (TSAN)** ima zadatak da uhvati bilo kakve Raise Conditional-e. Jedini problem u radu sa nitima je kada vise niti deli I menja podatke. U ovom slucaju postoji za skoro svaku nit posebna shadow memorija (celija) koja cuva:

- 1) TID (thread identifier)
- 2) vremenska oznaka kada je nit pokusala da pristupi odredjenom bitu u memoriji
- 3) flegove kojim tacno bajtovima je nit pokusala da pristupi
- 4) jedan bit koji govori da li je read ili write (r/w)

Za svakih 8 bajtova u originalnom memoriji ce biti 4 takve shadow celije.

Prilikom izvorsavanja programa, kada neka nit pokusa da pristupi odredjenom delu u memoriji kompajler ce da generise dodatne instrukcije da rade jos nesto I proveravace da li niti zele da pristupe istim delovima memorije (preko 3) stavke proverava da li postoji preklapanje) takodje gleda preko stavke 4) da li niti zele da pisu ili citaju. Kada se na taj nacin detektuju kolizije (potencijalni bagovi) potrebno je proveriti dodatne inf u vezi katanaca (da li su ti delovi memorije bili zakljucani, da li su stvari sinhronizovane kako treba ili ne).

U slucaju thread sanitizer-a koristi se znacajno vise memorije u odnosu na address sanitizer, a takodje I daje vece usporenje programu. Zato TSAN treba s vremena na vreme pokretati, a ne svaki put.

**Memory sanitizer (MSAN)** ima zadatak da detektuje da li pristupamo (u smislu citamo, jer mozemo da pisemo logicno) memoriji koja nije inicijalizovana. Ovde takodje imamo glavnu I shadow memoriju (1 bajt iz originalne odgovara 1 bajtu iz shadow memorije). Shadow memorija moze da bude **bit** maska sta je inicijalizovano u glavnom memoriji (1) a sta nije (0). Ako pokusamo da pristupimo tim zelenim flegovima (tj. jedinicama) onda nam shadow memorija kaze da je sve ok, a crvenim (tj. nulama) "ubijamo" program. Pamtimo za svaki bit a ne bajt zbog bitovskih operacija (zato sto moze biti  $a = a \mid 00100$ ), a drugi razlog je sto mozemo da kazemo  $\text{int } a:2$  (a je 2 bita),  $\text{int } b:4$ ,  $\text{int } c:2$  i onda ce kompajler ta 3 da zapakuje u 1 bajt, a neka od tih promenljivih moze da bude inicijalizovana a neka ne. Problem sa MSAN-om je sto kada deklarismo neku

strukturu koja se sastoji od char-ova i int-ova, a posto se u memoriji radi poravnanje na 8 bajtova, kada stavimo char ostaje neiskorisceni prostor i onda kompajler mora da zna da taj deo memorije nije bag vec tako treba.

domaci: ukucati man clang i man gcc i traziti /sanitizer i videti koji jos sanitajzeri postoje. Mnogi od njih ce pripadati UBSAN (Undefined Behaviour SANitizer) npr. detektuju prekoracenja prilikom kastovanja. Ono sto je bitno, ne treba ukljucivati vise sanitajzera odjednom vec 1 po 1 jer dosta njih nisu medjusobno kompatibilni.

## 9. nedelja, 20. decembar 2022.

- Na koji nacin mozemo da pisemo programe koji rade ispravno ono sto treba da rade?

- kako god pisemo programe, napravity se neka greska

- Na koji nacin testiramo programe?

- pisemo **testove**, ali iako testovi prolaze 100% to ne znaci da je nas program korektan jer mogu biti skriveni bagovi koji se **sanitajzerima** mogu otkriti. Takodje, ako smo ukljucili sanitajzere prilikom testiranja i testiranje prolazi 100% idalje nismo sigurno da li je nas kod korektan jer postoje bagovi koji se periodичno ispoljavaju (**flaky tests** – testovi koji su cudno napisani i nisu ponovljivi, desi se nekad da padnu). Pored svega toga, testovi su programi sto znaci “sve sto napisemo ce imati bag” => testovi ce imati bag. **UB** (undefined behaviour) - situacija u kojoj npr. pristupamo nekom pokazivacu koji ne pokazuje na objekat onog tima koji mi mislimo da pokazuje ili koji je null pokazivac, tada ako napisemo p -> f() to je UB. Kompajler u tom slucaju moze da radi nesto pametno, moze da radi nesto drugo pametno ali moze i da napravi katastrofalnu gresku.

**US** (unspecific behaviour) - ponasanje koje nije standardom propisano vec svaki kompajler radi na svoj nacin. Npr. f(a(b), b(a,c)) redosled izrsavanja a, b koje odredjen.

Zbog svega toga moze nas kompajler da radi savrseno, ali neki drugo kompajler ima drugacije ponasanje i nas program ne radi kako treba. Zbog toga testovi nikada nisu dovoljni, primer:

```
p -> f()
```

```
if(p){
```

```
    nesto...
```

```
}else{
```

```
    nesto...
```

```
}
```

=> problem sa ovim kodom je sto mi ovde dereferenciramo p i onda

kompajler moze da zakljuci ako vi ovde ne smete da dereferencirate p onda on nije null.

A da je pisalo if(p){ p -> f(a) } onda je to okej.

Obicno se takve “kreativnije” optimizacije nalaze na O2 nivou optimizacije, zato mnogi jezici ne dozvoljavaju optimizacije na nivou O3.

primer:

```
std::cin >> s;
```

```
//tained promenljiva – promenljiva koja moze da bude bilo sta i za
```

```
malloc(s*);           // koju je obavezna provera pre bilo kakvog kreiranja
sistemski_poziv(s);
```

primer:

```
key = ...;           //kljuc za enkripciju
int l = k % 2;
if(l) {
```

...

```
}           // ovo nije ni UB, ni US, ali je potencijalni problem jer imamo grananje koje zavisi
            od te "skinevene" promenljive. Njena vrednost moze da se kesira l sazna
```

To su sve bile dinamicke provere ispravnosti programa (program se izvrsava da bismo videli da li radi ispravno), pored njih postoje i **staticke analize programa**.

### ~ Staticka analiza ~

- nesto sto cita kod naseg programa i bez pokretanja vrsi analizu (pokusava da pronadje da li postoji bag). (npr. O3, -Wall, -Werror (proglasava sve warninge da budu greske), -pedantic)

Kompajler radi staticku analizu zbog:

- 1) optimizacije
- 2) da bi nam prijavio neke bagove
- 3) transformisanja koda

Nacini za staticku analizu:

1. grep i regularni izrazi (mozemo da napisemo osnovne provere, da ih opisemo reg izrazom koje ce blokirati osnovne probleme koji mogu da se dese) Mozemo da opisemo ovo npr:

```
if (a == 10) {...}
```

```
if (this) { double *d; d = ()malloc(42*sizeof(d))} ovde je problem sizeof(pokazivac)
```

2. bez parsiranja naseg koda: Codespell – program koji za bilo koji jezik proverava li smo dobro spelovali imena promenljivih (camel case)
3. učitavanje AST-a: ovo je dosta komplikovan proces. C++ nije jednostavan jezik zbog viseznacnosti izraza: npr. ovo `t * p` moze da znaci mnozenje, deklaracija pokazivaca (pokazivac `p` na tip `t`). Takodje, `t <42> (2)` moze da znaci da li je `t` manje od 42, pa to vece od 2, a moze da znaci da je `t` sablon funkcija sa netipskim argumentom i poziv te funkcije sa brojem 2. Takvi jezici su kontekstno-senzitivni jezici (imaju kontekstno-senzitivnu gramatiku).

Nijedan alat neće to implementirati od nule, već će koristiti kompajler **clang**. Kada je nastajao, ideja je bila da bude što je moguće više modularan (da delove možemo da koristimo gde god želimo).

1. 2. 3. su drugi nivo optimizacije

Treći (najviši) nivo optimizacije je simulacija nas dok razmišljamo kako naš program radi. Bez stvarnog pokretanja programa, analizator će pokušati da razmotri “šta ako sam otisao tamo...”. Na neki način, radi simulaciju izvršavanja našeg programa, svih mogućih stanja programa ali bez konkretnog izvršavanja. To je dosta problematično i komplikovano. Npr:

```
int x = 0;
if( flag) {
    x = 42;
    if (flag) {
        y = 1/x;
    }
}
```

Ovde jedino šta može da napravi nekakav problem je deljenje. Međutim x ne može da bude 0 jer da bismo usli u deo sa y, moramo da prodjemo dodelu x = 42. Ali, ako krene zaključivac liniju po liniju da analizira: na početku x = {0}, zatim x = {0, 42} a y može da bude šta god je definisano pre ili 1/x. To bi bilo false approval.

=> Najozbiljniji i najsporiji statički analizatori se trude da simuliraju sva stanja našeg programa i da za svaku granu pamte moguće vrednosti. Što više if-ova imamo, eksponencijalno će rasti broj stanja koja treba da izvršimo.

komentar: Alati su bazirani na ovim stvarima o kojima smo danas pričali.

Prvi alat **CppCheck** na nivou regularnih izraza. Može da “hvata” dosta jednostavne stvari.

Drugi alat **Clang-tidy** – može da nalizira kod napisan pre 10 godina i da preporuci kako da ga modernizujemo (u projektu ne treba nikako da uključimo sve flegove za modernize!!!!) Pored clang-tidy-a za Qt projekat postoji alat **clazy** (baziran na AST).

Predstavnik najvišeg nivoa optimizacije je alat **clang-analyzer**.

## 10. nedelja, 27. decembar 2022.

~ LCov, Hotspot, Git hooks ~

Pokrivenost (coverage) koda predstavlja proveru da li je svaka linija koda pokrenula, proverena tj. testirana. Za svaku liniju koda pokrivenost dodeljuje neki broj koji predstavlja koliko puta se konkretno ta linija izvršila prilikom izvršavanja programa. Pokrivenost možemo da računamo: 1)

ručno pokretanje i testiranje i 2) pokretanjem automatskih testova. Ako pokrenemo testove, onda taj broj predstavlja koliko puta se ta linija koda izvršila prilikom testiranja. A ako ne pokrenemo testove, taj broj će se razlikovati naravno, biće uključene samo one linije koda koje smo eksplicitno testirali. Znači kod 1) se pokrivenost razlikuje, a kod 2) se vrata uvek isti rezultat pokrivenosti. Svaki put kada pokrenemo program, velika je verovatnoća da smo ga drugačije testirali tako da će se i pokrivenost razlikovati. Prvenstveno, pokrivenost računamo kada izvršavamo automatske testove, za takva pokretanja, pokrivenost treba da bude više od 90%. Pokrivenost se obično izražava u procentima i uvek imamo neki donji limit koji želimo da bude ispunjen. Pokrivenost nikada nije 100% jer postoje obrade gresaka nekih delova u koje se nikad ni ne udje jer do greške ne dodje tako da se te linije koda ni ne testiraju. Pored tog procenta, imamo inf za svaku liniju koliko puta se izvršila. Naravno što je veći taj broj to bolje, jer to onda znači da postoji više testova koji pokrivaju tu funkciju.

Da bismo obavestili CMake da želimo da se računaju ti coverage izveštaji, potrebno je:

u build direktorijumu pozvati `cmake .. -DCMAKE_BUILD_TYPE=Debug -DCMAKE_CXX_FLAGS='-coverage' -DCMAKE_C_FLAGS='--coverage'`

A ako hocemo da vidimo eksplicitno koji si tu flegovi, dodamo: `-ftest-coverage` i `-fprofile-arcs`.

Kada taj cmake prodje, pozivamo `make -j10 VERBOSE=1`

Takvo pokretanje je generisalo fajlove koji u sebi sadrže inf koliko puta se nešto izvršilo. Ti fajlovi imaju ekstenziju `gcda` i postoji po jedan za svaki fajl izvornog koda. Te fajlove ne otvaramo direktno, već koristimo alat `LCov` koji će te fajlove analizirati i generisati neke izveštaje. Pokrenemo: `lcov --capture --directory --output-file coverage.info` i time se svaki `gcda` fajl analizira i formira se fajl `coverage.info` koji takođe nećemo direktno da otvaramo već od njega generisemo html fajl radi lepšeg slikovitog prikaza: `genhtml --output-directory coverage-html --legend coverage.info` time smo izvršili komandu i rekli gde želimo da se smeste ti html generisani fajlovi. Tu (u terminalu) dobijamo inf ukupnu pokrivenost linija koda, kao i funkcija (ne možemo da očekujemo veliki procenat ako je ručno testiranje u pitanju). Najzad, možemo da otvorimo u web browseru: `firefox -P EMPTY coverage-html/index.html`.

- U izveštaju se desava da neke funkcije imaju čudna imena, npr. ne prepozna da je u pitanju destructor za neku klasu, tako da komandom `c++filt 'čudno ime funkcije'` možemo da saznamo na koju se to tačno funkciju odnosi.

- Može se desiti da jedna funkcija koju smo napisali postane više funkcija u assembleru.

- Bolje je dobijati izveštaj o pokrivenosti na osnovu automatskih testova. U te svrhe, možemo da pokrenemo testove komandom `make check` (stari način). Postoji i komanda `ctest` koja dolazi uz cmake, možemo nju da pokrenemo. Kao i kod cmake, uz nju možemo da preciziramo koliko jezgara da koristi prilikom testiranja npr. `ctest -j4`. Nakon toga, pokrenemo `lcov` kao malopre i vidimo slikovite rezultate koji su sada bolji. Ne testiraju se GUI funkcije. Dobri projekti imaju već u `CMakeLists.txt` komande koje omogućavaju uključivanje `LCov` izveštaja, generisanje html itd.

- Kako su testovi registrovani?

Obicno se nalaze u direktorijumu tests. U CmakeLists.txt nalazi se funkcija enable\_testing() koja govori cmake-u da treba da registruje sve testove koje cemo kasnije pozvati. U tests direktorijumu, nalazi se opet neki CMakeLists.txt jer je to novi projekat.

**Hotspot** je alat koji omogucava da se prikazu informacije (performanse) za vec profilisan program pro.data, a takodje mozemo da snimimo podatke tj. pokrenemo program I onda sistem analizira koliko dugo se izvorsavao, koliko je koja funkcija oduzela vremena prilikom izvorsavanja I slicno. Profajleri su korisni kada nam se ponasanje menja u izvorsavanju programa, u nekim delovima krene da secka ili da se sporije izvrsava, pa pomocu njih mozemo da vidimo koje su to funkcije I da ih optimizujemo.

### Git Hooks

- odemo u .git I izvorsimo komandu ll, vidimo da postoje razni direktorijumi branches, hooks, info, logs, objects, refs. Otvorimo direktorijum hooks I u njemu imamo nekoliko definisanih fajlova koje git sam generise, a nismo ih mi direktno definisali. Hook-ove mozemo da tretiramo kao eventHandler-e za odredjene dogadjaje koji se desavaju tokom koriscenja git-a. Hook-ovi su funkcije kome mi pisemo I koje ce git izvorsavati u odredjenim situacijama. Npr. mozemo da napravimo git hook koji proverava da li je email adresa na projektu bas email adresa sa zvanicnog sajta fakulteta. Git hook su shell scripture tako da koristimo tu sintaksu:

```
#!/bin/sh
username=$(git config user.email)
if [ "$username" != "ivan@math.rs" ]
then
cat <<\EOF
Error: Mail nije ivan@math.rs
EOF
exit 1          //ako vratimo 0 sve okej, ako vratimo bilo sta drugo nesto nije dobro
fi
```

Nakon toga probamo da izvorsimo git commit I dobijemo poruku da Mail nije ...

- Git Hook-ove mozemo da pisemo I u npr. python-u (pravimo hook koji za fajlove koji su izmenjeni proverava da li su dobro formatirani ili ne)

```
#!/usr/bin/python3 ili #!/usr/bin/env python3
import subprocess //mora da se ukljuci taj modul za pokretanje nekih drugih programa
output = subprocess.check_output(['git', 'clang-format', '--diff']) // u shellu: var =
                                $(neka_komanda)
if 'clang-format did not modify any files'.encode() not in output and 'no modified files'.encode()
not in output:
print("Run git clang-format, git add, commit")
exit(1)
else:
exit(0)
```

Git hook-ovi se izvršavaju kod nas na lokalnu, pre nego što se izvrši commit, a CI se izvršavaju nakon pravljenja commit-a, nakon što smo naše podatke poslali na server. Kada se uradi git clone, ne kloniraju se git hook-ovi sa servera, što je malo mana jer onda svaki programer mora posebno sam za sebe lokalno da pravi hook-ove.

Mozemo da pisemo skripte za GDB:

Napisacemo skriptu koja radi novu jednostavnu komandu koja ce se zvati continue\_or\_run tj. znaci zelimo da spojimo 2 komande (run I continue) u jednu.

Takve stvari obicno stavljamo u poddirektorijumu direktorijuma home koji se zove .gdb:

```
mkdir -p ~/.gdb/my_py_scr      //opcija -p pravi 2 direktorijuma
```

```
vim ~/.gdb/my_py_sc/cont_or_run.py
```

```
class ContOrRun (gdb.Command):
```

```
def __init__(self):
```

```
super(ContOrRun, self).__init__('cont_or_run', gdb.COMMAND_USER)
```

```
def invoke(self, arg, from_tty):      //tty je arhaican naziv za Unix shell
```

```
if 0 < len(gdb.selected_inferior().threads()):    // ako je program pokrenut
```

```
gdb.execute('continue')
```

```
else:
```

```
gdb.execute('run')
```

```
ContOrRun()
```

provera u if-u: ako je duzina liste koja sadrzi pokrenute niti programa veca od 0 znaci program je pokrenut.

- Testiramo tu nasu komandu tako sto pokrenemo gdb I učitamo skrip koji smo pisali komandom  
source ~/.gdb/my\_py\_sc/cont\_or\_run.py

Nakon toga imamo komandu koja se zove samo cont\_or\_run, pritisnemo enter I tako je mozemo koristiti kad god hocemo.